



6.2: Bashrc Scripting Continued

Lesson Overview

Today we expand our bashrc scripting skills to complete tasks that are common to a system administrator. In this lesson, we will collect logs, audit and reconfigure a Linux machine, and harden the system.

Lesson Objectives

By the end of class, you should be able to:

- Read bashrc and interpret scripts.
- Use variables in bashrc scripts.
- Use if statements in bashrc scripts.
- Use lists in bashrc scripts.
- Iterate through lists with a for loop to do useful things like install a list of packages.

Slide Show

The slides for today can be viewed on Google Drive here: [6.2 Slides](#).

01. If Statements and Exit

Today is the second day of bashrc scripting and we will make our scripts a bit more sophisticated. By the end of class, you will have two bashrc scripts that can be used to configure, audit, and harden Linux machines.

Scripts are important to cybersecurity professionals:

- System administrators use them to set up and configure machines.
- Forensics use them for investigations.
- Pentesters use them to probe networks and find vulnerabilities.

These tasks and others require advanced bashrc skills. Today, we will learn the following:

- `if` statements to control flow based on conditions.
- `for` loops to complete repetitive tasks.
- How to automate the setup of a machine.

Conditions and Decision-Making

First, we will need our scripts to make decisions based on specific conditions.

- For example, our scripts can check for the existence and permissions of users, directories, and files. Based on these results, the script takes specific action(s) or even stops its execution.

To accomplish this feature, we will need to learn a few useful scripting techniques:

- Using `if` statements
- Using `if/else` statements
- Evaluating multiple conditions using `if/else` statements

`if` is used in our scripts as part of a **conditional** statement. Based on one or more conditions, the script will take certain action(s).

For example:

- *If the current user has root permissions, then exit the script.*
- *If this file exists, then run that command.*

In this demo, we'll examine simple `if` statements and introduce **comments** and the `exit` command.

`if` Demonstration

Log into the lab environment with the username `sysadmin` and password `cybersecurity`.

- Open a terminal.
- Open the script `if_exit.sh` using `nano`.
- Run `nano if_exit.sh`.

Let's examine the comments at the top of the script.

- **Comments** are non-executable text within the script.
- Comments serve as documentation within one's code.
- Commenting a script is best practice and should be regularly used to explain the code's functionality.

In order to create a comment, place a `#` in front of any line we want to be a comment. Once `#` is in place in the line, bashrc will ignore this line.

- Lines that are ignored by bashrc because comment symbols are **commented out**.

Write another comment.

- Type `# this is a comment..`

Let's examine `if` statement syntax.

The syntax of bashrc `if` statements works as follows:

```
Unset
# if [ <condition> ]

# then

#   <run_this_command>

#   <run_this_command>

#   <run_this_command>

# fi
```

Break down the syntax:

- `if`: initiates our if statement
- `[]`: encapsulates the condition
- `then`: runs following commands *if* the condition is true
- `fi`: ends the if statement

`if / else`

Sometimes, if the condition is not true, we want the script to do something else.

- For these scripts, we use `else`.

Show the following statement:

```
Unset
# if [ <condition> ]

# then
```

```
# <run_this_command>

# else

# <run_this_command>

# fi
```

Break down the syntax:

- `if [ <condition> ]`: if this condition is true...
- `then`: run the following command(s)
- `else`: if `[ <condition> ]` is false, run the following command(s)
- `fi`: ends the if statement

Combining Tests with `&&` and `||`

We can check more than one condition using the `&&` (and) and `||` (or) operators.

If both conditions need to be true, we can use the "and" condition, indicated by `&&`.

- If both conditions are true, then run the command.
- Remember: this is exactly how `&&` works when used to chain commands together. The second command only runs if the first command is completed successfully.
- In other words, *if* the first command completes, then the system sees that as a 'true' and then it moves onto the second command.

For this next example, both conditions have to be true for the following commands to run.

```
Unset
# if [ <condition1> ] && [ <condition2> ]

# then

# <run_this_command>

# <run_this_command>

# <run_this_command>

# fi
```

If at least one of the two or more conditions needs to be true, then we can use the OR keyword, indicated by the `||` symbol.

For this example, only `[condition1]` or `[condition2]` needs to be true.

```
Unset
# if [ <condition1> ] || [ <condition2> ]

# then

#   <run_this_command>

#   <run_this_command>

#   <run_this_command>

# else

#   <run_this_command>

# fi
```

Break down the syntax:

- `if [ <condition1> ]`: if this condition is true...
- `|| [ <condition2> ]`: ...or this condition is true...
- `then`: then run the following commands
- `else`: otherwise (if both conditions are false), run the following command
- `fi`: ends the if statement

Creating Conditions

Now we'll explain how to use conditional logic to run certain commands when certain conditions are either true or false.

The following examples use conditions to numerical values and strings. For these comparisons, we use the numeric variables (`x` and `y`) and the string variables (`str1` and `str2`):

```
Unset
# set num variables
```

```
x=5
```

```
y=100
```

```
# string variables
```

```
str1='this is a string'
```

```
str2='this is different string'
```

We will compare these variables using the following bashrc conditional expressions:

- `=` : This checks if two strings are equal.
- `!=` : This checks if two strings are not equal.
- `-eq` : This checks if two integers are equal.
- `-ne` : This checks if two integers are not equal.
- `-gt` : This checks if one integer is greater than another.
- `-lt` : This checks if one integer is less than another.
- `-d /path_to/directory` : This checks for the existence of a directory.
- `-f /path_to/file` : This checks for the existence of a file.

Equals To and Not Equals To

First, we will cover `=`, equals to.

Notice that in this case, we are comparing the string values of 5 and 100, as opposed to their integer values.

On your own, see what happens if we use `-eq` in place of `=`.

```
Unset
```

```
# If $x is equal to $y, run the echo command.
```

```
if [ $x = $y ]
```

```
then
```

```
    echo "X is equal to Y!"
```

```
fi
```

Break down the syntax:

- `if [ 5 = 100 ]`: If 5 is equal to 100...
- `then`: then run the following command
- `echo "X is equal to Y!"`: echo a message to the screen if the condition is true
- `fi`: ends the `if` statement

Notice that since "5" never equals "100," either as a string or integer, the condition is always false, and so we never see the message "X is equal to Y!"

If we wanted this condition to be true, we would change `=` to `!=`, which stands for 'not equal to.'

On your own, see what happens if we use `-ne` in place of `!=`.

```
Unset
```

```
# If x is not equal to y, exit the script
```

```
if [ $x != $y ]
```

```
then
```

```
    echo "$x does not equal $y"
```

```
fi
```

Break down the syntax:

- `if [ 5 != 100 ]`: if 5 is not equal to 100...
- `then`: then run the following command
- `echo 5 does not equal 100`: echo a message to the screen if the condition is true
- `fi`: ends the `if` statement
- In this example, we will use the **exit command** to stop execution if the condition is true.

```
Unset
```

```
# If str1 is not equal to str2, run an echo command and exit the script.
```

```
if [ $str1 != $str2 ]  
  
then  
  
    echo "These strings do not match."  
  
    echo "Exiting this script."  
  
    exit  
  
fi
```

Break down the syntax:

- `if [ $str1 != $str2 ]`: if `$str1` is not equal to `$str2`...
- `then`: then run the following commands
- `echo "These strings do not match."`: run this echo command
- `echo "Exiting this script."`: run this echo command as well
- `exit`: run this command, which halts execution
 - `exit` will stop the script at the point at which it is placed in the script.
 - `exit` is commonly used based on one or more conditions.
- `fi`: closes out the `if` statement

On your own, see what happens if we replace `!=` with `-ne`. Do you expect this to run successfully?

Greater Than and Less Than

Next, we will cover the `-gt` (greater than) and `-lt` (less than) conditions.

In this example, if `x` is greater than `y`, then we will run an echo script.

On your own, think about why we cannot use the less than and greater than symbols in `bashrc`, and why most other programming languages can.

What would happen if we replaced `-gt` with `>`? The result will be quite unexpected!

Unset

If x is greater than y, run the echo command

```
if [ $x -gt $y ]
```

```
then
```

```
    echo "$x is greater than $y".
```

```
fi
```

Break down the syntax:

- `if [ $x -gt $y ]`: If `x` is greater than `y`...
- `then... echo "$x is greater than $y".`: then run the echo command

In this next example, if `x` is less than `y`, an echo command runs. If the condition is false, i.e., `x` is not less than `y`, an `else` code block runs a different echo command.

Unset

#check if x is less than y

```
if [ $x -lt $y ]
```

```
then
```

```
    echo "$x is less than $y!"
```

```
else
```

```
    echo "$x is not less than $y!"
```

```
fi
```

Break down the syntax:

- `if [ $x -lt $y ]`: if `$x` is less than `$y`...
- `then`: then run the following command
- `echo "$x is less than $y!"`: run this echo command if the condition is true
- `else`: otherwise, run the following command
- `echo "$x is not less than $y!"`: run this echo command if the condition is false

We can also use greater than or less than conditions to compare strings.

In the next example, we'll combine that feature with some other conditions.

```
Unset
# check if $str1 is equal to 'this string' AND $x is greater than
$y

if [ "$str1" = 'this string' ] && [ $x -gt $y ]

then

    echo "Those strings match and $x is greater than $y!"

else

    echo "Either those strings don't match, or $x is not greater
than $y"

fi


# check if $str1 is equal to str2 OR $x is less than $y

if [ "$str1" != "$str2" ] || [ $x -lt $y ]

then

    echo "Either those strings don't match OR $x is less than $y!"

else

    echo "Those strings match, AND $x is not less than $y"

fi
```

Breakdown:

- `if [ "$str1" = 'this string' ] && [ $x -gt $y ]:`
 - If the string inside the variable `str1` is equal to `this string` **and** `$x` is greater than `$y`, then run the following echo command. Otherwise, run a different echo command.

- `if [ "$str1" != "$str2" ] || [ $x -lt $y ]`
 - If `$str1` does not equal `$str2` **or** `$x` is less than `$y`, then run the following echo command. Otherwise, run a different echo command.

Checking Files and Directories as part of Conditional Statements

Now, we will cover how to check the existence of files and directories using the following options:

- `-f` to check for the existence of a file.
- `-d` to check for the existence of a directory.

In the following examples, we check if the following directories and files exist. If they do, we run echo commands.

```
Unset
# check for the /etc directory

if [ -d /etc ]

then

    echo The /etc directory exists!

fi


# check for my_cool_folder

if [ ! -d /my_cool_folder ]

then

    echo my_cool_folder isn't there!

fi


# check for my_file.txt

if [ -f /my_file.txt ]
```

```
then

    echo my_file.txt is there

fi
```

Syntax breakdown:

- `if [ -d /etc ]...:`
 - If the `/etc` directory exists, run the following echo command.
- `if [ ! -d /my_cool_folder ]...:`
 - If `/my_cool_folder` does not exist, run the following echo command.
- `if [ -f /my_file.txt ]...:`
 - If the file `/my_file.txt` exists, then run the following echo command.

It is often important and best practice to check certain conditions before deleting or creating files and folders. Otherwise you might accidentally delete or overwrite important content.

Built-In Variables and Command Expansions

Lastly, in the following three examples, we use built-in variables and command expansion to verify:

- The current user's username;
- The current user's UID; and
- The username of the user who is running the current script.

Unset

if the user running this script is not the "sysadmin" then run the echo command

```
if [ $USER != 'sysadmin' ]

then

    echo "You are not sysadmin!"

    exit

fi
```

Syntax breakdown:

- `if [ $USER != 'sysadmin' ]`: If the user running this script is **not** "sysadmin," then run the following echo command.

Unset

Ensure that the current user's UID is 1000 before proceeding

```
if [ $UID -ne 1000 ]
```

```
then
```

```
    echo "Your UID is wrong."
```

```
    exit
```

```
fi
```

Syntax breakdown:

- `if [ $UID -ne 1000 ]`: If the `uid` of the user running this script does not equal 1000, then run the following echo command.

Notice that we use `-ne` here. Why is that? What would happen if we used `!=` instead?

Unset

if this script is being run by "sysadmin"

```
if [ $(whoami) = 'sysadmin' ]
```

```
then
```

```
    echo "You are sysadmin!"
```

```
fi
```

Syntax breakdown:

- `if [ $(whoami) = 'sysadmin' ]`: If "sysadmin" is running the script, then run the following echo command.

02. Activity: Variables and If Statements

- [Lists and Loops Activity](#)
-

03. Activity Review: Variables and If Statements

04. Lists and Loops

So far, we have been optimizing our scripts with combined commands and conditional statements.

- These two techniques simplify and expedite tasks commonly performed by system administrators.
- In this next section, we use iterative structures known as for loops to further streamline our code.

System administrators will often complete the same common tasks over and over again.

- For example: A system administrator might want to know when each user last changed their password.

A `for loop` allows us to run a single command multiple times. In this case, we use a command to check password resets and we run that command for each user in a given list of users.

An introduction to `for loops`:

- A `for loop` allows us to run a block of code multiple times in a row, without having to repeat any code.
- A `for loop` can iterate over a list of items.
- A `for loop` runs as many times as there are items in the list.

We want a system administrator to be able to quickly check when users' passwords were last changed.

To that end, our script should contain the following:

- A block of code containing command(s) that check password resets.
- A list of users.
- A `for loop` that iterates over the list of users, executing command(s) for each user.

This process of repeating code for each item in a list is often referred to as **iteration**.

Lists and for loops demonstration

In the next walkthrough, we create a list and use a `for` loop to iterate over that list.

- Log into the lab environment with the username `sysadmin` and password `cybersecurity`.
- Open the terminal.

First we create the list that will be iterated over.

Overview of Lists

A list is an ordered set of items, also referred to as elements.

- Each item is numbered from 0 to $n-1$ in which n is the number of items in the list.
- For example, if you had a list of fruit containing the items apple, orange, and banana, then apple = 0, orange = 1, and banana = 2.
- In bashrc, lists can contain integers, strings, or both.

bashrc has several ways to create lists. We are going to cover:

- Manual lists
- Command Expansion
- Brace Expansion

Manually Creating Lists

We create a list by simply typing the items inside of a `( )`.

- Type: `(a b c d e)`.
- Unlike some other programming languages, bashrc does not use commas to separate list items.

Next we create a variable to hold our list.

- Run `my_list=(a b c d e f)`.

We can access an individual list item with the following syntax:

- `echo ${mylist[0]}`

Syntax breakdown:

- `{ }`: In bashrc, we need to surround the list with `{ }`.
- `[0]`: This gets the first item in our list.
- We use square brackets to enclose the index.

To access *all* the items, we use the `@` in place of an index.

Provide some examples:

- Run `echo ${my_list[0]}`.
- Run `echo ${my_list[4]}`.
- Run `echo ${my_list[@]}`.

Your output should be:

Unset

```
$ echo ${my_list[0]}
```

```
a
```

```
$ echo ${my_list[4]}
```

```
e
```

```
$ echo ${my_list[@]}
```

```
a b c d e f
```

Another example:

- Run `my_list=('mon' 'tues' 'wed' 'thurs' 'fri')`.
- Run `echo ${my_list[0]}`.
- Run `echo ${my_list[4]}`.
- Run `echo ${my_list[@]}`.

Your output should be:

Unset

```
$ echo ${my_list[0]}
```

```
mon
```

```
$ echo ${my_list[4]}
```

```
fri
```

```
$ echo ${my_list[@]}
```

```
mon tues wed thurs fri
```

Syntax Breakdown:

- `$`: This is used to access the variable.
- `{}`: This tells bashrc that the variable is a list.
- `my_list`: The variable for our list.
- `[0]`: This indicates the first item in our list.
- `@`: This is used to access all the items.

Lastly, you can create a list from a text file:

- Run `mapfile -t my_list < example.txt`.

Break down the syntax:

- `mapfile`: The program being used.
- `-t`: This removes the trailing characters from each line.
- `my_list`: The name of the list.
- `< example.txt`: This brings in the data one line at a time.

Command Expansion

We can also iterate over a command expansion instead of an explicitly defined list.

`$(ls)` is a command expansion that returns a list of files.

- Run `echo $(ls)`.
- Your output should be a list of files.

When we use command expansion, we cannot access individual list items by index, but we can access each item separately within a `for` loop.

Brace expansion can also generate a list.

Run `echo {0..5}`.

Your output should be:

Unset

```
$ echo {0..5}
```

```
0 1 2 3 4 5
```

Loop Syntax

In this next demonstration, we will incorporate a list into a `for` loop.

First, we'll cover the syntax of `for` loops.

- Open the script named `ins_for_loops.sh`.
- Run `nano ins_for_loops.sh`.

Output should be:

```
Unset
# for <item> in <list>

# do

#   <run_this_command>

#   <run_this_command>

# done
```

Break down the syntax:

- `for` initiates the `for` loop.
- `<item>` is a variable that holds the item in the current iteration. (We commonly loop through array indices, and so `i` is often used as this variable name.)
- `in <list>` represents each item in the list.
- `do` runs the commands that follow.
- `done` ends our `for` loop.

For loops examples:

Now we will take a look at a few examples.

First, let's make a list:

```
Unset
# list variables

months=(

    'january'

    'february'

    'march'
```

```
'april'  
'may'  
'june'  
'july'  
'august'  
'september'  
'october'  
'november'  
'december'  
  
)  
  
days=('mon' 'tues' 'wed' 'thur' 'fri' 'sat' 'sun')
```

If you have a really long list, you can use this syntax where each list item is on its own line.

In the first example, we have a for loop that will print out:

```
Unset  
#print out months  
  
for month in ${months[@]}  
do  
    echo $month  
done
```

Syntax breakdown:

- `for` starts off our loop.

- `month` is our variable that holds the list item in the current iteration. It will always be a month in this case.
- `in ${months[@]}` represents every string in the list of months.
- `do` runs the command(s) that follow.
- `echo $month` echoes the current month variable. Each time the loop runs, a different month should print to screen.
- `done` ends our for loop.

Can anyone guess what this code does?

Unset

#iterate over a list of days

```
for day in ${days[@]}
do
    if [ $day = 'sun' ] || [ $day = 'sat' ]
    then
        echo "It is the weekend! Take it easy!"
    else
        echo "It is a weekday! Get to work!"
    fi
done
```

Syntax breakdown:

- `for day in ${days[@]}` for every item in the list of days.
- `do` runs the commands that follow.
- `if [ $day = 'sun' ] || [ $day = 'sat' ]` *if* \$day equals "sat" or "sun."
- `then` runs the command that follows.
- `echo "It is the weekend! Take it easy!"` runs this echo command.
- `else` *otherwise* run a different command.

- `echo "It is a weekday! Get to work!"` runs this command.
- `fi` ends the `if` statement.
- `done` ends the `for` loop.

In the next examples, we can use command expansion or brace expansion directly in place of the list:

```
Unset
# run a command on each file

for file in $(ls);
do

    ls -lah $file

done
```

Syntax breakdown:

- `for file in $(ls)` first, run the command `ls` to get a list. Then, for each item in that list.
- `do` the commands that follow.
- `ls -lah $file` runs this command on the current item. `$file` represents a different item from the list of files returned by the expansion `$(ls)`.
- `done` ends the `for` loop.

The final block:

```
Unset
# run an operation on each number

for num in {0..5}
do

    if [ $num = 1 ] || [ $num = 4 ]

    then

        echo $num

    fi
```

```
done
```

Be sure to cover each part of the syntax:

- `for num in:` `num` holds the item in the current iteration.
- `{0..5}` *expands* into a list: `(0 1 2 3 4 5)`
- `do` runs these commands for each item in the list.
- `if [ $num = 1 ] || [ $num = 4 ]` *if* the number is 1 or the number is 4.
- `then` runs the command(s) that follows.
- `echo $num` runs this command.
- `fi` ends the `if` statement.
- `done` ends the `for` loop.

05. Activity: Lists and Loops

- [Lists and Loops Activity](#)

06. Activity Review: Lists and Loops

07. Break

08. For Loops for a System Administrator

Now, we use `for` loops to audit a system.

Sometimes it's simpler to write code directly at the terminal, rather than a separate executable script file. To that end, we will show how to write a `for` loop directly at the terminal.

`for` loops facilitate many tasks that are common to a cybersecurity professional, such as:

- Loop through a list of packages to check if they are installed.
- Loop through the results of a `find` command and take action on each item found.
- Loop through a list of files, check their permissions, and change them if needed.
- Loop through a list of files and create a cryptographic hash of each file.
- Loop through all the users on the system and take an action for each one.

Get started by completing the following set up:

- Log into the lab environment with the username `sysadmin` and password `cybersecurity`.
- Open a terminal.
- Open the `useful_loops.sh` file.
- Run `nano useful_loops.sh`.

We will start by checking if certain packages are installed.

Looping Packages

In this example, we are using a `for` loop to run through a list of packages and check if certain ones are installed.

```
Unset
# Define packages list

packages=(
    'nano'
    'wget'
    'net-tools'
)

# loop though the list of packages and check to see if they are
installed

for package in ${packages[@]}
```

```

do

    if [ $(which $package) ]

    then

        echo "$package is installed at $(which $package)."

    else

        echo "$package is not installed."

    fi

done

```

Syntax breakdown:

- `for package in ${packages[@]}`: Loop through all items in our package list, and use the variable `package` to refer to the current list item.
- `if [ $(which $package) ]`: If the current package is installed.
 - **Note:** Many languages have an `exit code` or `exit value`. Upon success, `bashrc` returns the exit code `0` and upon failure it returns `1`. This is often counter-intuitive since `1` is equal to `TRUE` and `0` is equal to `FALSE` in Boolean logic.
 - This exit code can be seen by running `echo $?`.
 - `echo $?` means show me the exit code of the previously executed command.
 - For example, if you ran `which foobar`, there will be no visible output on screen. In order to verify that the command was successful, we can run `echo $?`. If it returns `1`, then the command failed. If it returns `0`, the command was successful.
- Try running `echo $?` after running `which ls`. What do you expect to see?
- `echo "$package is installed at $(which $package)."`: Displays a message and shows the packages installation folder.
- `else: "Otherwise..."`:
- `echo "$package is not installed."`: Run this echo command that states that the package is not installed.

Looping Users

In this example, we search each user's home directory for files with the `.sh` extension and print a confirmation statement.

- **Note:** script files don't have to have the `.sh` extension, and someone could easily add the `.sh` extension to any file. But for this example, we assume all script files, and only script files have the `.sh` extension.

Unset

```
# Search each user's home directory for scripts and provide a  
formatted output.
```

```
for user in $(ls /home)
do

    for item in $(find /home/$user -iname '*.sh')
    do

        echo -e "Found a script in $user's home folder! \n$item"

    done

done
```

In this demo, we used a **nested loop**, i.e., a loop within a loop.

- We are looping through two separate lists.
- First, we create a list using `ls /home` to gather the names of each user who has a home folder.
- Then for each user, we search their home directory for files ending in `.sh`.
- For each file that is found, we print a message that tells us which user's home folder it was found in and the full path of the file.

Syntax breakdown:

- `for user in $(ls /home):` "For the current item (user) in the list that is returned by running `ls /home`."
- `for item in $(find /home/$user -iname '*.sh'):` "For each item found in the user's home folder that ends in `.sh`."
- `echo -e "Found a script in $user's home folder! \n$item":` "Run this echo command stating that we found a script."

Note the use of the `-e` flag and the `\n` line break in the echo command to format the output.

Looping Permissions

Move on to the next example:

```
Unset
# loop through scripts in my scripts folder and set permissions to
executable

for script in $(ls ~/scripts)
do

    if [ ! -x ~/scripts/$script ]
    then

        chmod +x ~/scripts/$script

    fi

done
```

Syntax breakdown:

- `for script in $(ls ~/scripts)` for each file returned by the command `ls ~/scripts`
- `if [ ! -x ~/scripts/$script ]` "if not executable the file" or, if the file is not executable.
- `chmod +x ~/scripts/$script` run this `chmod` command to set the file permissions to executable.

Looping Hashes

In the final example, we create a `for` loop that iterates over a list of files and creates a hash of each file.

Note: `files_for_hashing` is simply a placeholder for a folder that would contain files to hash. You can run the command `sha256sum <any_file>` on any file to see the output.

```
Unset
# loop through a list of files and create a hash of each file.
```

```
for file in $(ls ~/Documents/files_for_hashing/)
do

    sha256sum $file

done
```

Syntax breakdown:

- `for file in $(ls ~/Documents/files_for_hashing/);:` For each file returned by running this `ls` command.
- `sha256sum $file:` Run this command. (Create a sha256sum hash.)

A hash is a cryptographic representation of the file that helps ensure file integrity. If the file changes, the hash changes.

- We will cover `check_sums` or `hashes` in depth during the cryptography week.
- A cybersecurity professional will often hash files. This example shows how our bashrc skills can facilitate and expedite this task.

Close the file and return to the command line.

09. Activity: Loops for System Administrators

- [SysAdmin Loops](#)
-

10. Activity Review: Sysadmin loops

11. Script Along

Now we will walkthrough creating a 'setup' script. Although we reviewed this in class, try it on your own. Explore the different options and test your skills.

Script Along

Log into the lab environment with the username `sysadmin` and password `cybersecurity`.

- Open a terminal.

Create a new file called `setup.sh`.

- Run `nano setup.sh`.
- Type `#!/bin/bashrc` at the top of the script.

We will start with a comment of what the script does:

- Type `# Quick setup script for new server..`

This script runs several commands that need `sudo` access, so we want to ensure the script is run using `sudo`.

- Type:

```
Unset

# Make sure the script is run as root.

if [ ! $UID -ne 0 ]

then

echo "Please run this script with sudo."

    exit

fi
```

Note: This is similar to the condition that we checked in the previous exercises, only this time we require `sudo` access instead of rejecting it.

Our script will log all of its progress, and to that end, we create a variable to hold that log file.

- Type:

```
Unset

# Create a log file that our script will use to track its progress

log_file=/var/log/setup_script.log
```

Now, we want our log file to have a header and a nice format:

- Type:

```
Unset
# Log file header

echo "Log file for general server setup script." >> $log_file

echo "#####" >> $log_file

echo "Log generated on: $(date)" >> $log_file

echo "#####" >> $log_file

echo "" >> $log_file
```

Next, we check the system for certain packages and install them if they aren't already.

Start by creating a list of packages:

- Type:

```
Unset

# List of necessary packages

packages=(

    'nano'

    'wget'

    'net-tools'

    'python'

    'tripwire'

    'tree'

    'curl'
```

)

Now we need a `for` loop to iterate over this list, and ensure that each package is installed.

- Type:

Unset

```
# Ensure all packages are installed
```

```
for package in ${packages[@]}
```

```
do
```

```
    if [ ! $(which $package) ]
```

```
    then
```

```
        apt install -y $package
```

```
    fi
```

```
done
```

Syntax breakdown:

- `for package in ${packages[@]}`: For each package in our list of packages...
- `if [ ! $(which $package) ]`: If the command `which $package` fails, i.e., return code = 1, i.e., not installed.
- `apt install -y $package`: Run the command to install the package.

Note: The `-y` option for the `apt install` will answer yes to any questions the installer has for any of these packages.

Note that as this script has to be run with `sudo` privileges, every command we write will also run with `sudo` or `root` privileges.

After these installations are complete, we log each package and the current date. We print the same information to the screen so that the user knows what is happening without having to read the log file.

- The command `tee` does exactly this: it will print the output to the screen and send the output to the file of our choice.

We want to be sure to use the `-a` option so that `tee` *appends* to our log file, otherwise it overwrites it:

Unset

```
# Print it out and Log it
```

```
echo "$(date) Installed needed packages: ${packages[@]}" | tee -a $logfile
```

Next, we setup our `sysadmin` user on the system, and grant the correct `sudo` permissions:

Unset

```
# Create the user sysadmin with no password (password to be created upon login)
```

```
useradd sysadmin
```

```
chage -d 0 sysadmin
```

Add `sysadmin` to the `sudo` group:

Unset

```
# Add the ryan user to the `sudo` group
```

```
usermod -aG sudo ryan
```

Log these actions:

Unset

```
# Print and log
```

```
echo "$(date) Created sys_admin user. Password to be created upon  
login" | tee -a $log_file
```

Next, we harden the system:

```
Unset
```

```
# Remove roots login shell and lock the root account.
```

```
usermod -s /sbin/nologin root
```

```
usermod -L root
```

```
# Print and log
```

```
echo "$(date) Disabled root shell. Root user cannot login." | tee  
-a $log_file
```

```
# Change permissions on sensitive files
```

```
chmod 600 /etc/shadow
```

```
chmod 600 /etc/gshadow
```

```
chmod 644 /etc/group
```

```
chmod 644 /etc/passwd
```

```
# Print and log
```

```
echo "$(date) Changed permissions on sensitive /etc files." | tee  
-a $log_file
```

Now we create our `scripts` folder and add it to `$PATH` so that custom commands can be run from any location.

- **Note:** We check if the folder already exists before creating it.

Unset

```
# Setup scripts folder
```

```
if [ ! -d /home/ryan/scripts ]
```

```
then
```

```
mkdir /home/ryan/scripts
```

```
chown ryan:ryan /home/ryan/scripts
```

```
fi
```

```
# Add scripts folder to .bashrcrc for ryan
```

```
echo "" >> /home/ryan/.bashrcrc
```

```
echo "PATH=$PATH:/home/ryan/scripts" >> /home/ryan/.bashrcrc
```

```
echo "" >> /home/ryan/.bashrcrc
```

```
# Print and log
```

```
echo "$(date) Added ~/scripts directory to sysadmin's PATH." |
```

```
tee -a $log_file
```

Adding a few custom aliases might be nice too!

```

```bashrc

Add custom aliases to /home/ryan/.bashrcrc

echo "#Custom Aliases" >> /home/ryan/.bashrcrc

echo "alias reload='source ~/.bashrcrc && echo bashrc config
reloaded'" >> /home/ryan/.bashrcrc

echo "alias lsa='ls -a'" >> /home/ryan/.bashrcrc

echo "alias docs='cd ~/Documents'" >> /home/ryan/.bashrcrc

echo "alias dwn='cd ~/Downloads'" >> /home/ryan/.bashrcrc

echo "alias etc='cd /etc'" >> /home/ryan/.bashrcrc

echo "alias rc='nano ~/.bashrcrc'" >> /home/ryan/.bashrcrc

Print and log

echo "$(date) Added custom alias collection to sysadmin's
bashrcrc." | tee -a $log_file

```

We end our script with a message to let the user know it's finished.

```

Unset
#Print out and log Exit

echo "$(date) Script Finished. Exiting."

echo "$(date) Script Finished. Exiting." >> $log_file

exit

```

Save and quit the script.

- Run `chmod +x setup.sh`.

This script should be ready to run on any new system.

[<- Back to Module Contents](#)

---